

DEVOPS INTERVIEW PREP SERIES

Shell Scripting Deep Dive

Simple English · Visual Notes · Interview-Ready Answers

PART 20 — Shell Scripting Deep Dive

Bash Basics | Text Processing | Practical SRE Scripts | Cron Jobs

4.1 Bash Scripting Basics

What does a well-structured bash script look like?

```
#!/bin/bash
# Shebang - specifies the interpreter

# Script metadata
# Author: ...
# Date: ...
# Purpose: Monitor disk usage

# Variables
THRESHOLD=80
EMAIL="admin@example.com"

# Functions
check_disk() {
    local mount_point=$1
    usage=$(df -h "$mount_point" | awk 'NR==2 {print $5}' | sed 's/%//')
    echo "$usage"
}

# Main logic
main() {
    usage=$(check_disk "/")
    if [ "$usage" -gt "$THRESHOLD" ]; then
        echo "Disk usage is ${usage}% - exceeds threshold!"
    fi
}

# Execute
main
```

Good structure: shebang first, then metadata comments, then variables, then functions, then a clear `main` that ties everything together and is called at the very end.

Variables, command substitution, and special variables.

```
NAME="Irfan"
AGE=23
readonly PI=3.14159      # constant, cannot be changed later

echo "Hello ${NAME}"     # preferred - clear variable boundary

CURRENT_DATE=$(date +%Y-%m-%d) # command substitution (modern)
FILES_COUNT=`ls -l | wc -l`    # old-style backticks

export PATH=$PATH:/opt/bin
echo $HOME; echo $USER; echo $PWD
```

Special Variable	Meaning
\$0	Script name
\$1, \$2, ...	Positional arguments
\$#	Number of arguments
\$@	All arguments, as separate words
\$*	All arguments, as one single word
\$\$	Current process's PID
\$?	Exit status of the last command
\$_	PID of the last background process

Conditional statements and comparison operators.

```
if [ "$AGE" -gt 18 ]; then
    echo "Adult"
elif [ "$AGE" -eq 18 ]; then
    echo "Just turned adult"
else
    echo "Minor"
fi
```

Numeric

-eq, -ne, -gt, -ge, -lt, -le

String

=, !=, -z (empty), -n (non-empty)

File Tests

-f (file), -d (dir), -e (exists), -r/-w/-x (perms), -s (not empty)

```
if [ -f "/etc/passwd" ]; then echo "File exists"; fi
if [ -z "$VAR" ]; then echo "Variable is empty"; fi

# Logical operators
if [ "$AGE" -gt 18 ] && [ "$COUNTRY" = "USA" ]; then echo "Can vote"; fi
if [ "$AGE" -lt 18 ] || [ "$STUDENT" = "yes" ]; then echo "Discount"; fi

# [[ ]] - extended test, supports regex directly
if [[ "$EMAIL" =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email"
fi

# case statement
case "$OS" in
    linux) echo "Linux system" ;;
    darwin) echo "macOS" ;;
    *) echo "Unknown OS" ;;
esac
```

Loop types in bash.

```
# For loop - iterate a list
for i in 1 2 3 4 5; do echo "Number: $i"; done

# For loop - C style
for ((i=1; i<=5; i++)); do echo "Count: $i"; done

# For loop - files
for file in /var/log/*.log; do echo "Processing $file"; done

# For loop - command output
for user in $(cat /etc/passwd | cut -d: -f1); do echo "User: $user"; done

# While loop
count=1
while [ $count -le 5 ]; do echo "Count: $count"; ((count++)); done

# While - read a file line by line
while IFS= read -r line; do echo "Line: $line"; done < input.txt

# Until loop
count=1
until [ $count -gt 5 ]; do echo "Count: $count"; ((count++)); done

# break and continue
for i in {1..10}; do
    if [ $i -eq 5 ]; then continue; fi    # skip 5
    if [ $i -eq 8 ]; then break; fi      # stop at 8
    echo $i
done
```

Functions, return values, and local vs global variables.

```
greet() {  
    echo "Hello $1"  
}  
  
# Function "returning" a value via echo + capture  
add() {  
    local sum=$(( $1 + $2 ))  
    echo $sum  
}  
result=$(add 5 3)  
  
# Function with a return CODE (0-255, not a real value)  
check_file() {  
    if [ -f "$1" ]; then return 0; else return 1; fi  
}  
if check_file "/etc/passwd"; then echo "File exists"; fi  
  
# Local vs global  
global_var="I'm global"  
my_function() {  
    local local_var="I'm local"    # only visible inside this function  
    global_var="Modified global"   # modifies the outer variable  
}
```

4.2 Text Processing

grep — pattern matching reference.

```
grep "error" /var/log/syslog
grep -i "error" file.txt      # case insensitive
grep -v "debug" file.txt     # invert match
grep -c "error" file.txt     # count matches
grep -n "error" file.txt     # show line numbers
grep -r "TODO" /path/to/code # recursive
grep -l "error" *.log        # show only filenames with matches

grep -A 3 "error" file.txt    # 3 lines after
grep -B 3 "error" file.txt    # 3 lines before
grep -C 3 "error" file.txt    # 3 lines before and after

grep -E "error|warning|critical" file.txt # extended regex
grep -w "log" file.txt         # word boundary - matches "log" not "login"
grep -f patterns.txt input.txt # multiple patterns from a file
```

sed — stream editor reference.

```
sed 's/old/new/' file.txt      # first occurrence per line
sed 's/old/new/g' file.txt    # all occurrences
sed 's/old/new/gi' file.txt   # global, case-insensitive

sed -i 's/old/new/g' file.txt  # edit in-place
sed -i.bak 's/old/new/g' file.txt # edit in-place with backup

sed '/pattern/d' file.txt      # delete matching lines
sed '2,5d' file.txt           # delete lines 2-5

sed -n '10p' file.txt         # print only line 10
sed -n '/pattern/p' file.txt  # print only matching lines

sed '5i\New line' file.txt    # insert before line 5
sed '5a\New line' file.txt    # append after line 5

sed 's/foo/bar/g; s/baz/qux/g' file.txt # multiple operations

# Replace using regex capture groups
echo "2024-01-15" | sed 's/\([0-9]\{4\}\)-\([0-9]\{2\}\)-\([0-9]\{2\}\)/\3\/\2\/\1/'
# Output: 15/01/2024
```


awk — pattern scanning and processing reference.

```
awk '{print $1}' file.txt           # first column
awk '{print $1, $3}' file.txt      # first and third
awk '{print $NF}' file.txt         # last column

awk -F: '{print $1}' /etc/passwd  # custom field separator

awk '/error/ {print $0}' file.txt  # lines matching a pattern
awk '$3 > 100 {print $1, $3}' file.txt # numeric comparison

# Built-ins: NR (line number), NF (field count), FS, OFS, RS

awk 'NR==5' file.txt              # print line 5
awk 'END {print NR}' file.txt      # total line count

awk '{sum += $3} END {print sum}' file.txt      # sum a column
awk '{sum += $3} END {print sum/NR}' file.txt    # average
awk '{if ($3 > max) max = $3} END {print max}' file.txt # maximum

awk '{printf "%-10s %5d\n", $1, $2}' file.txt  # formatted output

awk '/error/ {count++} END {print "Errors:", count}' /var/log/syslog
```

cut, sort/uniq, and tr — quick reference.

```
# cut - extract columns
cut -c1-5 file.txt           # characters 1-5
cut -d: -f1,3 /etc/passwd   # fields 1 and 3, colon-delimited

# sort & uniq
sort file.txt                # alphabetical
sort -n file.txt             # numeric
sort -k 2 file.txt           # sort by column 2
sort file.txt | uniq -c      # count occurrences of each line
sort file.txt | uniq -d      # only lines that appear more than once

# tr - translate characters
echo "Hello" | tr '[:lower:]' '[:upper:]' # HELLO
echo "Hello123" | tr -d '[:digit:]'       # Hello
echo "Hello  World" | tr -s ' '           # squeeze repeated spaces
```

4.3 Practical SRE Scripts

1. Disk Usage Monitor

```
#!/bin/bash
# disk_monitor.sh - Alert when disk usage exceeds threshold

THRESHOLD=80
EMAIL="admin@example.com"
HOSTNAME=$(hostname)

check_disk() {
    df -H | grep -vE '^Filesystem|tmpfs|cdrom' | awk '{print $5 " " $1}' | while read
output; do
        usage=$(echo $output | awk '{print $1}' | sed 's/%//g')
        partition=$(echo $output | awk '{print $2}')
        if [ $usage -ge $THRESHOLD ]; then
            echo "ALERT: Disk usage on $partition is ${usage}% on $HOSTNAME"
        fi
    done
}
check_disk
```

2. Log Analyzer

```
#!/bin/bash
# log_analyzer.sh - Analyze log files for errors

LOGFILE="/var/log/application.log"
REPORT="/tmp/log_report_$(date +%Y%m%d).txt"

analyze_log() {
    echo "Log Analysis Report - $(date)" > $REPORT
    error_count=$(grep -c "ERROR" $LOGFILE)
    echo "Total Errors: $error_count" >> $REPORT

    echo "Top 10 Error Messages:" >> $REPORT
    grep "ERROR" $LOGFILE | awk '{print $5,$6,$7,$8,$9}' | sort | uniq -c | sort -rn |
head -10 >> $REPORT

    echo "Errors by Hour:" >> $REPORT
    grep "ERROR" $LOGFILE | awk '{print $2}' | cut -d: -f1 | sort | uniq -c >> $REPORT

    cat $REPORT
}
analyze_log
```

3. Service Health Check

```
#!/bin/bash
# health_check.sh - Check service health and restart if needed

SERVICES=("nginx" "mysql" "redis")
LOG="/var/log/health_check.log"

check_service() {
    local service=$1
    if systemctl is-active --quiet $service; then
        echo "$(date): $service is running" >> $LOG
        return 0
    else
        echo "$(date): $service is DOWN - Attempting restart" >> $LOG
        systemctl restart $service
        sleep 5
        if systemctl is-active --quiet $service; then
            echo "$(date): $service restarted successfully" >> $LOG
        else
            echo "$(date): CRITICAL - $service failed to restart" >> $LOG
        fi
        return 1
    fi
}

for service in "${SERVICES[@]}; do
    check_service $service
done
```

4. Backup Script (with rotation)

```
#!/bin/bash
# backup.sh - Automated backup with rotation

SOURCE="/var/www"
DEST="/backup"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="backup_${DATE}.tar.gz"
RETENTION_DAYS=7

tar -czf ${DEST}/${BACKUP_NAME} $SOURCE 2>/dev/null

if [ $? -eq 0 ]; then
    echo "Backup completed: ${DEST}/${BACKUP_NAME}"
    find $DEST -name "backup_*.tar.gz" -type f -mtime +$RETENTION_DAYS -delete
    echo "Old backups cleaned up (retention: $RETENTION_DAYS days)"
else
    echo "ERROR: Backup failed"
    exit 1
fi

if [ -f "${DEST}/${BACKUP_NAME}" ]; then
    size=$(du -h "${DEST}/${BACKUP_NAME}" | cut -f1)
    echo "Backup size: $size"
fi
```

5. Process Monitor

```
#!/bin/bash
# process_monitor.sh - Monitor CPU/Memory usage of processes

THRESHOLD_CPU=80
THRESHOLD_MEM=80

monitor_processes() {
    echo "Process Monitoring - $(date)"

    echo -e "\nTop 5 CPU consumers:"
    ps aux --sort=-%cpu | head -6 | awk '{printf "%-10s %5s %5s %s\n", $1, $3, $4, $11}'

    echo -e "\nTop 5 Memory consumers:"
    ps aux --sort=-%mem | head -6 | awk '{printf "%-10s %5s %5s %s\n", $1, $3, $4, $11}'

    echo -e "\nProcesses exceeding thresholds:"
    ps aux | awk -v cpu=$THRESHOLD_CPU -v mem=$THRESHOLD_MEM '
        NR>1 {
            if ($3 > cpu || $4 > mem) {
                printf "PID: %-6s CPU: %5s%% MEM: %5s%% CMD: %s\n", $2, $3, $4, $11
            }
        }'
}
monitor_processes
```

4.4 Cron Jobs

Cron syntax and common scheduling examples.

```
* * * * * command
| | | | |
| | | | | — Day of week (0-7, 0 and 7 are Sunday)
| | | | — Month (1-12)
| | | — Day of month (1-31)
| | — Hour (0-23)
| — Minute (0-59)
```

```
crontab -e    # edit crontab
crontab -l    # view crontab
crontab -r    # remove crontab

*/5 * * * * /path/to/script.sh      # every 5 minutes
0 * * * * /path/to/script.sh        # every hour
30 2 * * * /path/to/backup.sh        # every day at 2:30 AM
0 0 * * 0 /path/to/weekly_task.sh    # every Sunday at midnight
0 0 1 * * /path/to/monthly_task.sh   # 1st of every month
0 9 * * 1-5 /path/to/workday_task.sh # Monday-Friday at 9 AM
*/15 9-17 * * 1-5 /path/to/script.sh # every 15 min, business hours
0 6,12,18 * * * /path/to/script.sh  # multiple times a day

# Redirect output to a log
0 0 * * * /path/to/script.sh >> /var/log/cron.log 2>&1
```

Cron special strings.

Shortcut	Equivalent
@reboot	Run once at startup
@yearly	0 0 1 1 *
@monthly	0 0 1 * *
@weekly	0 0 * * 0
@daily	0 0 * * *
@hourly	0 * * * *

```
@daily /path/to/daily_backup.sh
@reboot /path/to/startup_script.sh
```